# Owner-approved repository remediation

## What the owner button does

With `compose.github.yaml` enabled on top of the Kubernetes overlay, the button is **Approve & create PR**. It records approval of the exact persisted manifest diff, then creates a branch, commit and pull request through the repository-scoped GitHub App. It does not merge the PR or patch the cluster.

```
Real pod events + immutable repository configuration
  -> ADK diagnosis -> fresh AI reviewer -> bounded manifest diff
  -> owner approval -> PR -> human merge -> merge-commit CI
  -> namespace-scoped deployment -> three consecutive readiness checks
  -> verified closure OR explicit failure/configuration rollback
```

The four Kubernetes examples remain real. The original eight synthetic scenarios keep their existing simulation behavior.

## Which access token belongs in the browser?

Use the application's **OWNER_TOKEN** from the private local configuration to approve or reject. Use its **OPERATOR_TOKEN** to start diagnosis. These are role credentials, not GitHub tokens. Never paste either into chat.

Do not enter a GitHub PAT, Copilot credential, App ID, installation ID or PEM key into the application's Access token box. The GitHub App key is mounted read-only on the backend; installation access tokens are generated server-side and kept only in memory.

After changing roles, the selected incident stays selected and the reason is cleared. The button is enabled only in an owner session with a reason of at least eight trimmed characters and a persisted repository diff. Backend authorization and state checks remain authoritative.

## One-time publication prerequisite

The application reads `kubernetes/examples.yaml` from this repository's **main** branch at a pinned commit. It never substitutes a local working copy. The integration code, dependency metadata, workflow and demo manifest must first be published through a reviewed setup PR and manually merged after CI succeeds.

Until then, **Repository-backed deployment** shows a clear blocked status. The old four live-only proposals cannot be approved in GitHub mode: they have no repository evidence or diff. Reject those legacy proposals as the owner, then start a fresh diagnosis as an operator after setup is merged. Do not reset the failing workloads just to create a new diagnosis.

Bootstrap publication is separate from incident remediation. The setup PR may contain application, test and workflow files; subsequent automated remediation PRs may modify only `kubernetes/examples.yaml`, and only one approved field in one of the four named Deployment documents.

# Local configuration

Prerequisites: Docker Desktop Kubernetes, the existing restricted service account, and a GitHub App installed only on `DataScience-KnowledgeBase/remedi-ai-copilot`. App repository permissions are Contents/Pull requests read-write and Actions/Checks read-only. The App cannot bypass branch protection and has no workflow-write or administration permission.

Run from the project root, using the private key's actual path:

```
$env:REMEDI_GITHUB_APP_ID = "5483871"
$env:REMEDI_GITHUB_INSTALLATION_ID = "175563857"
$env:REMEDI_GITHUB_PRIVATE_KEY_FILE = "C:\Users\Kura Dheeraj\.remedi-secrets\remedi-ai-config-agent.2026-10-26.private-key.pem"
docker compose -f compose.yaml -f compose.kubernetes.yaml -f compose.github.yaml up -d --build --wait
```

This is a local working-tree build, not evidence of a successful GitHub release. The key stays outside Git and is excluded from the model's evidence. The repository is public by the owner's explicit choice. Inspect every proposed diff before approving: no secrets may appear in manifests, PR bodies or commits.

For Actions deployment, configure these **repository variable values**, not secrets:

| Variable | Value |
|---|---|
| `REMEDI_KUBERNETES_CREDENTIALS_DIR` | Absolute local `.runtime\kubernetes` directory |
| `REMEDI_GITHUB_PRIVATE_KEY_FILE` | Absolute external private-key file path, not its contents |
| `REMEDI_GITHUB_APP_ID` | `5483871` |
| `REMEDI_GITHUB_INSTALLATION_ID` | `175563857` |

`REMEDI_LOCAL_ENV_FILE` continues to point to the private application configuration. Only the local Windows runner reads the private files. Hosted CI receives no real credentials or cluster access. Never upload the PEM as a repository variable or artifact.

# Walkthrough

1. Connect as an operator and check **Repository-backed deployment** is READY.
2. Select one Kubernetes example and click **Diagnose live pods**.
3. Inspect actual pod evidence, selected repository configuration, immutable source commit, model recommendation, reviewer result and the proposed diff. YAML formatting is normalized; semantic validation allows only the bounded field edit.
4. Connect with the application's owner token. Review the change and rollback plan, enter a reason and click **Approve & create PR**.
5. The status becomes `awaiting_merge`. Open the displayed PR link. There is no cluster write at this stage.
6. Review the diff and passing PR checks in GitHub; manually merge it yourself. The application never merges or enables auto-merge.
7. The status becomes `awaiting_ci`. The backend polls approximately every ten seconds and requires the trusted GitHub Actions CI check on the exact merge commit to succeed. Missing, pending, failed, cancelled,

skipped or spoofed checks do not authorize deployment.
8. After CI succeeds, the backend's bounded deployment adapter applies only the approved field, validated against the merged manifest. It does not run model-generated commands or apply an arbitrary repository YAML document.
9. Watch `executing` and `verifying`: actual replica counts, pod identity, readiness and consecutive checks are streamed. Three consecutive healthy samples on the same current-revision pod close the incident.

The existing Actions pipeline still tests manifest-only merges, but skips rebuilding the application deployment for those merges. This avoids restarting the controller that is waiting to deploy the remediation. Application-code changes still use the normal gated Docker deployment.

Process examples sequentially. A concurrent change to main, PR head, target template or approved configuration can invalidate a proposal; reanalyse instead of weakening the gate or automatically rebasing an approved diff.

## Failure, rollback and recovery boundaries

- Missing GitHub credentials, permissions or remote manifest fail explicitly. There is no direct-patch fallback in repository-backed mode.
- Repository changes after diagnosis can require fresh analysis.
- A changed PR head, extra changed files, nonhuman merge or differing merged manifest blocks deployment.
- Deployment uses an atomic UID/resourceVersion-tested bounded patch, only if the live pod-template checkpoint is unchanged.
- A rollout timeout can restore the approved configuration field if no external template change occurred. This does **not** mean recovery: the original failing configuration is restored. A human-reviewed repository revert PR is then required to resolve desired-state drift. It is not silently auto-reverted.
- Network failures or restarts after patch submission may leave a change applied. Execution/verification is not blindly replayed. Check the cluster and audit.
- Waiting-for-merge/CI states survive restarts. Deterministic incident branches and exact-content validation make interrupted PR creation resumable without duplicate commits or pull requests.
- Human merge waits have no automatic deadline. Token expiry, API errors and closed-unmerged PRs are explicit failures, not successful deployment.

This pilot still uses shared application role tokens, not independently attributed enterprise human identity. Repository rules require PRs, successful CI and manual merge without bypass; no second approving reviewer is required for the current single-maintainer setup.

## Validation scope

Mock GitHub/Kubernetes tests cover authentication, exact manifest edits, approval and legacy-proposal gates, idempotent PR creation, manual merge and trusted CI. Mock browser tests click the actual owner button and verify its request and waiting/error states without creating a real PR or changing a cluster.

No test result alone establishes a real PR merge/deployment. Those steps require the owner's actual approval and manual merge; the application leaves them pending.